TERRAFORM

1. **What is Terraform:** Terraform is an open source IAC (infrastructure as a code) tool by harshcorp, it is used to provision and manage the infrastructure in various cloud platforms like aws, azure, Gcp.

2. **What are the blocks we have in Terraform:** In Terraform we have different blocks like, Provider block, Resource block, variable block, backend block, connection block, datasource block like that.

3. **What are the terraform commands:** Yeah! The basic terraform commands like Terraform init→ Terraform validate→Terraform plan→Terraform apply→Terraform Destroy like that.

4. **Difference between count and foreach :** count is for creating resources by number (list, index-based), while for_each is for creating resources from maps/sets with unique configurations (key-value based).

5. **Statefile in Terraform :** Terraform statefile stores all the state information about the infrastructure. It's like the memory of the terraform helps to track the existing resources and what are the changes needed to do. By default the statefile will store in the local machine which is not secure.

6. **Then how to secure the statefile :** yeah!.. For storing our state file securely we should use the backend block while creating the infra. We should mention the remote bucket in the backend block. Based on this our state file is stored securely in the remote storages.

7. **Terraform lifecycle :** Yeah!.. Terraform lifecycle we have depends on, Ignore changes, Createbeforedestroy, PreventBeforeDestroy. Each lifecycle works in diff use cases,

8. **Depends on** → The resource creates one depending on the other. For eg : if i go to create the resource in the east region and west region. I mentioned it depends on the lifecycle in the east region (depending on west) this will perform like once the west is created the only the east will create the resource.

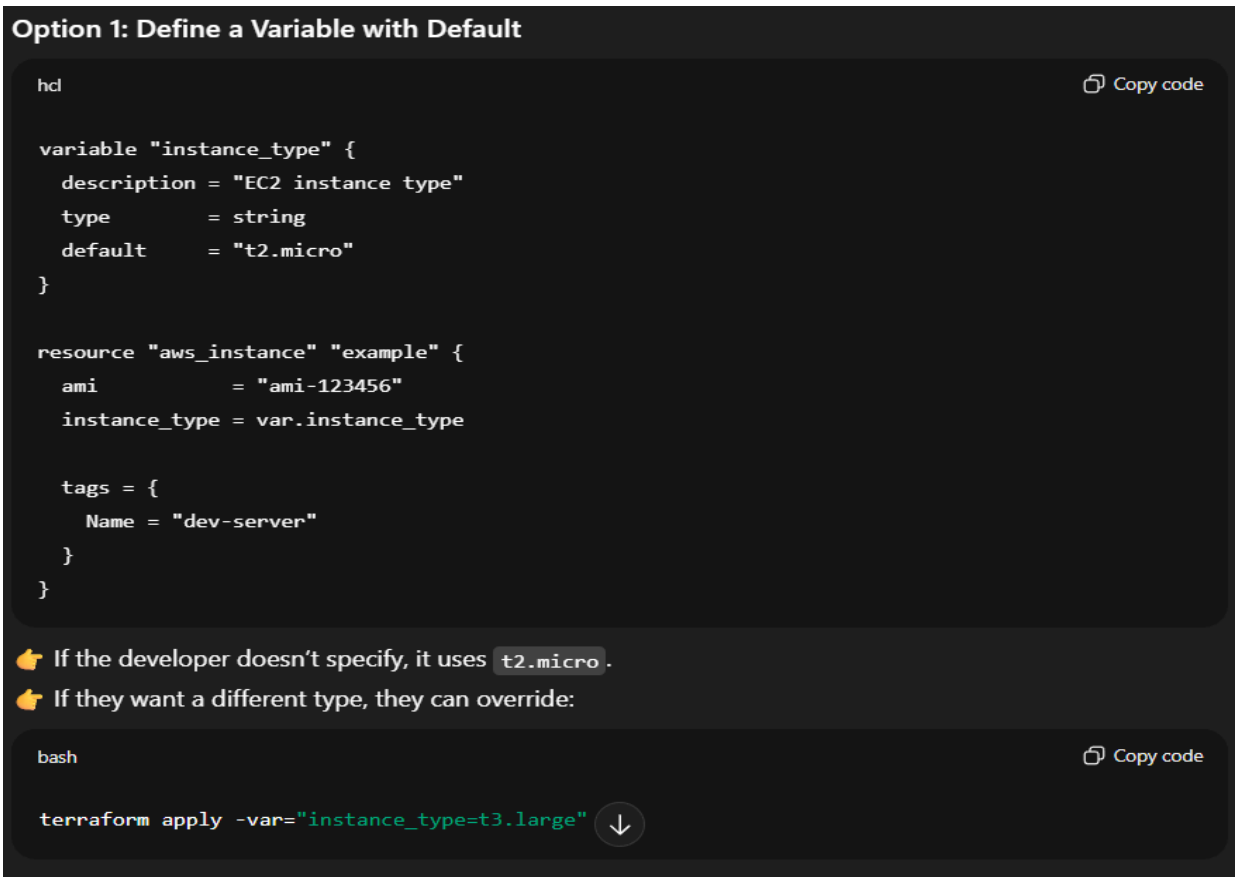
9. **IgnoreChanges** → It's like we created the resource in the terraform but we made some changes in the console. It doesn't store in statefile it just ignores the changes.

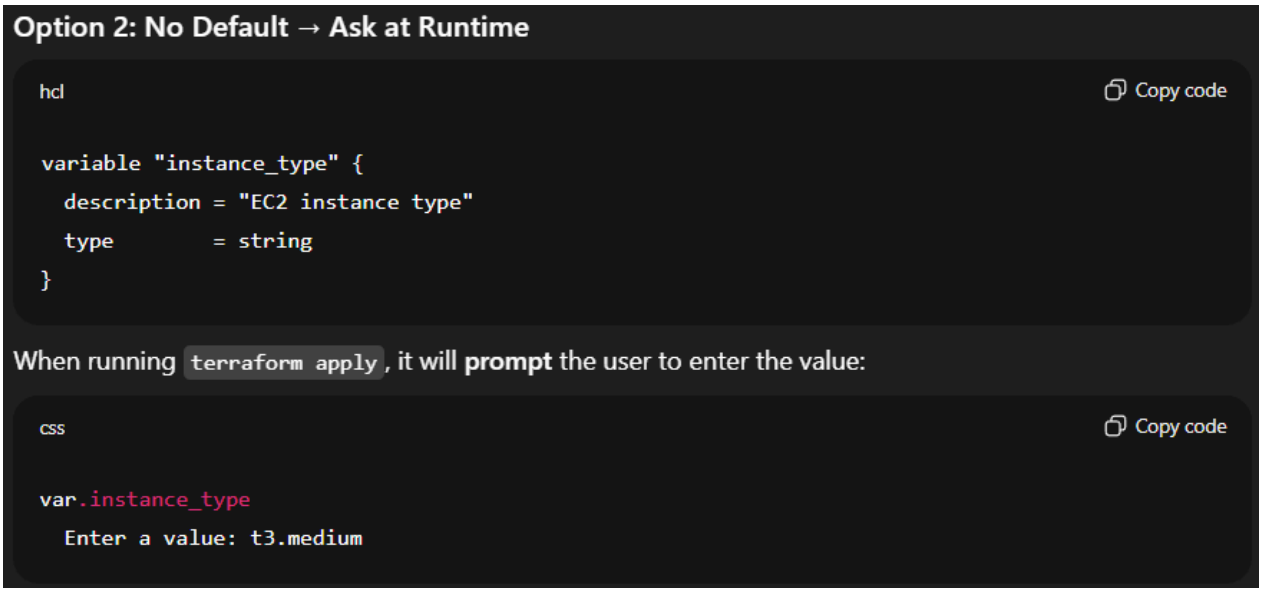
10. **CreateBeforeDestroy:** Basically the terraform behave first delete the resource and create the new one based on the terraform file. But in our lifecycle, If we mentioned the createbeforedestroy lifecycle inside the resource block, First it will create the new resource and then destroy the existing one.

11. **PreventBeforeDestroy:** If we mentioned the lifecycle preventbeforedestroy= True inside the resource block, when we run the destroy command, It will prevent the resource we are unable to destroy that particular resource.

12. **Which variable have a high persistent :** The High persistent would be for variable in CLI like **terraform apply --var instance_family=t3.micro (but i gave t2.micro in variable file**), Then the second presistent for Environment variable which we have set in the machine, Then the last is Terraformfile variable.
    Variable in CLI→ Environment variable→ Terraformfile.

13. **For example, your developer asks to create a server but he doesn't know the instance type:** If a developer doesn't know the instance type, use Terraform variables (with default, runtime prompt, or environment-based maps) so the value can be supplied later without changing the code.





14. **infra reusable and easier to manage across environments:**
    We can store all the environment variables in one file **file.tfvars** for different environments like prod, dev, test . So they can easily call the environment variables from main.tf.
    terraform plan -var-file="dev.tfvars"

15. **Terraform workspace and how to use what are the benefits of that:** We can maintain the isolated environment and each workspace would contain the individual state file so we can easily manage and access.

16. **Workspace:** create isolated workspaces for different environments which dev,prod,qa. why we create a workspace to maintain a state file separately for different environments.

- commands:

- terraform workspace show (current WS)
- terraform workspace list (shows all WS)
- terraform workspace new dev (Create new WS)

- terraform workspace delete dev (delete old WS)
- terraform workspace select prod (switch workspace)

17. **What is a Provisioner in Terraform :** A provisioner in Terraform is a way to execute scripts or commands on a resource after it's created or before it's destroyed.Think of it like: "I want Terraform to set up some configuration on this machine after it launches."

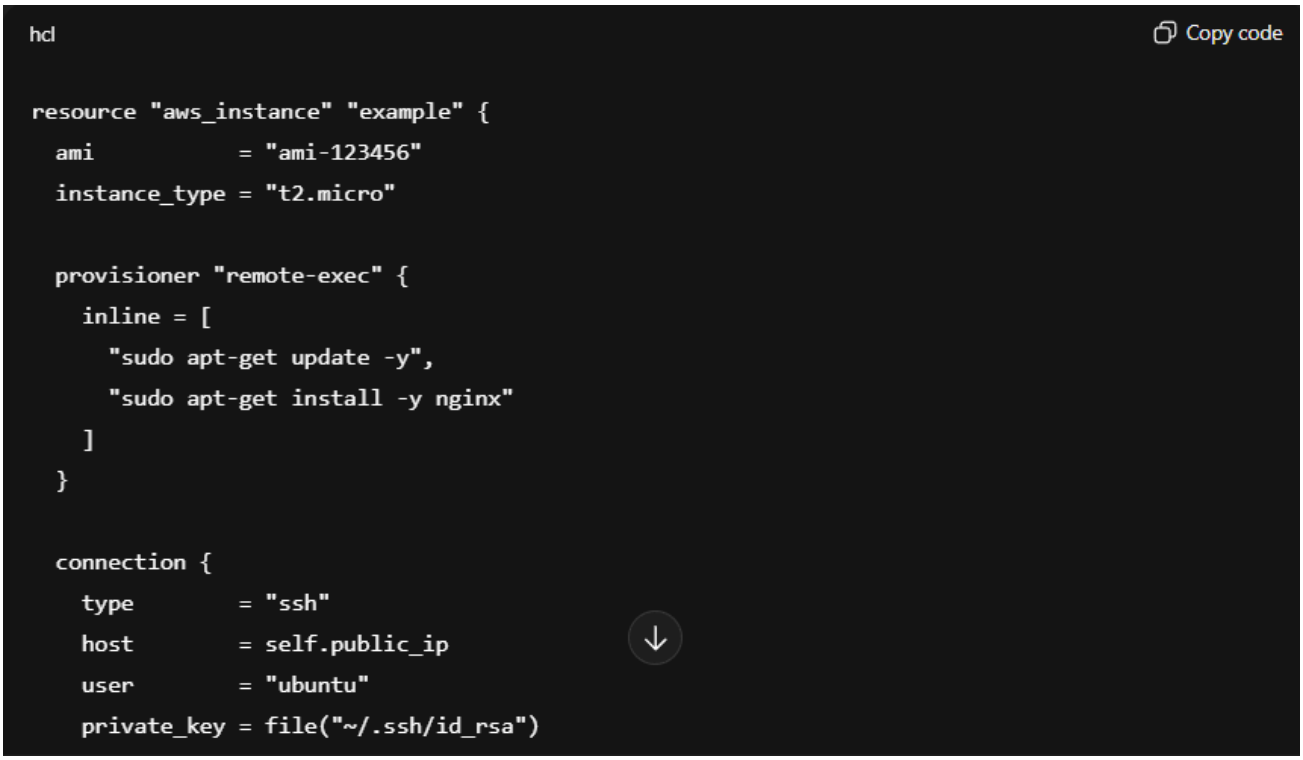
18. **What are the provisioners we have in Terraform :** We have a file provisioner, Local exec provisioner and Remote Exec provisioner.

    File Provisioner → If you want to copy a file from your local machine to a remote server while running a terraform script with the help of a connection block.

    Local exec provisioner → whenever we create the remote resource with the help of terraform script after that the local file will be executed.

    Remote Exec provisioner → whenever we create the remote resource with the help of terraform script after that the exec file will be executed in the remote server with the help of a connection block.

19. **Which provisioners need a connection block :** Only the file and remote provisioner have a connection block because to connect the remote resource from the local.



20. **Modules in terraform :** Module is like a helm chart. Modules are used to reuse the scripts. We split all the blocks like resource.tf, Environment.tf, providers.tf like that we can directly call the files in the main.tf file.

→ We can split the resource codes individually like ec2 directory, vpc directory like that.

🔹 Real-World Module Structure

Projects usually look like this 👇

css

Copy code

```
project-root/
├── main.tf              # Calls modules
├── variables.tf
├── outputs.tf
├── terraform.tfvars     # Environment-specific variables
├── modules/
│   ├── ec2/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── vpc/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── s3/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
```